CLAUDE SKILL ▪ RELEASE NOTES

# academic-pdf-reading

Version 1.3.0 | 11 October 2026

**At a glance**

| | |
|---|---|
| **What it is** | An error-averse protocol that makes Claude read academic papers and textbooks without silently corrupting their mathematics. |
| **When it runs** | Whenever you ask about an equation, derivation, figure, page, section or notation in a paper or textbook you uploaded or keep in a Project. |
| **What it ships** | `SKILL.md`, a reference on project protocols, and `triage.py`, a deterministic measurement script. |
| **Requires** | Claude with skills and code execution enabled. Python 3.8+; poppler-utils and Pillow are used when present. No network access. |
| **Install** | Customize → Skills (Section 4). |

## 1 Overview

Text extraction from scientific documents fails *silently*. A minus sign vanishes, a superscript $-1$ comes out as a subtraction, an overbar disappears, a prime turns into the digit 0, and calligraphic and roman letters merge. The result still looks like plausible mathematics, so neither the reader nor Claude notices that the formula being discussed is not the one on the page.

academic-pdf-reading gives Claude a fixed procedure that prevents a damaged text layer from reaching an answer as if it were the document's content. Claude uses the text layer to *find* things and reads everything mathematical from the *page image*. Every reading fact carries its provenance, and anything that could not be verified is labelled as such.

The skill triggers on its own when Claude is asked to explain, quote, check or transcribe material from a paper or textbook; when a file named `.pdf` turns out not to be a PDF; and when you ask Claude to create, check or update a project reading protocol (Section 2.7).

## 2 What the skill does

### 2.1 Core policy

These rules hold for every file and every question:

- **Two channels, two jobs.** The text channel is for locating content and for prose. The image channel is for mathematics, figures, tables, and anything whose meaning depends on position or typeface.
- **No mathematics from the text layer.** No sign, index, exponent, bar, hat, tilde, prime, dagger or typeface distinction is quoted from extracted text. It is read from the page image, cropped and upscaled where needed. If a file has no image channel, its mathematics is labelled *"text-layer reading, unverified"*.
- **Source outranks rendering.** If the LaTeX source or a pdf-to-markdown transcription of a document is available, Claude reads that instead.
- **Provenance on every fact.** Readings are marked *verified at p. N* (checked on the image) or *provisional* (inferred from context).

- **Printed citations.** Claude cites the printed page and the equation tag, and determines how the printed page relates to the page index in the file.
- **No guessing.** If a symbol is still ambiguous after cropping, Claude cross-checks it elsewhere in the document or says it is ambiguous.
- **Nothing transfers between files.** A glyph mapping learned in one document is never applied to another, even one from the same authors, journal or book.

## 2.2 Container triage

Claude identifies each file by its leading bytes, never by its extension, because a file called `.pdf` can arrive in several different forms:

| Signature | Container | Handling |
|---|---|---|
| `%PDF-` | genuine PDF | Text via `pdftotext`, images via `pdftoppm`. |
| `PK\x03\x04` | ZIP archive | Usually a paged archive (one JPEG plus one text file per page); unpacked into a writable directory. |
| UTF-8 text | text-only extraction | No image channel: all mathematics is labelled unverified. |
| transcription legend | pdf-to-markdown output | Read as source (Section 2.6). |
| `.tex`, `.md`, … | source | Ground truth; quoted as-is. |
| JPEG / PNG | page image | Image channel only. |
| anything else | unknown | Claude stops and reports instead of guessing. |

## 2.3 Text-layer failure modes

For each file with a text layer, the skill checks for three documented failure modes. They can co-occur, in which case the restrictions of all of them apply.

| Mode | What goes wrong | Typical signature |
|---|---|---|
| **A Substitution** | A re-encoded font makes glyphs extract as other, legible characters, for example letters standing in for parentheses or braces. | Unusual Latin-1 letters where delimiters belong; equation tags in the substituted form. |
| **B Geometry loss** | The characters are right but their positions are lost: superscripts read as arithmetic, accents detached or dropped, typefaces merged. | Many one- or two-character lines; displayed equations split across lines. |
| **C OCR** | The text layer was recognised from a scan, so it is probabilistic. | OCR font or producer strings; a full-page image on every page. |

The script flags suspects, and Claude confirms each one by comparing a displayed equation, an inline formula and a prose paragraph against the page image before relying on it.

## 2.4 Page numbering

Printed page numbers often differ from the page index in the file. `triage.py` detects folio patterns (journal e-locators, "*N of T*", header and footer numbers), reports the dominant offset and the pages that disagree, and flags front matter or appendices with a different offset. Claude confirms the offset on the page images before citing printed pages.

## 2.5 Workflow: locate, read, quote

1. **Locate** in the text channel: grep a distinctive prose word, or an equation tag in whatever form this particular file uses.

2. **Read** on the image channel: open the page and crop every line that carries a sign, index or accent, upscaled 2.5–4×.
3. **Quote** prose from the text layer only after deterministic repairs, and never under OCR. Quote mathematics only from the image, the source or a transcription, and cite as *Author Year, p. N, Eq. (X)*.

## 2.6 Reading pdf-to-markdown transcriptions

A document converted with the companion skill **pdf-to-markdown** arrives as `NAME.md` plus `NAME_figures.pdf`. Claude recognises the transcription by the legend on its first line. Because every formula in it was transcribed from page images and machine-checked when it was made, Claude reads it as source: it quotes the transcription as-is, cites it as *transcription, p. N*, and does not re-verify its equations against page images in the chat. For figure content, Claude opens the indicated page of the figures PDF.

Claude escalates instead of correcting silently in three cases: the transcription conflicts with a derivation or a companion paper; it is internally inconsistent; or the profile reports a structural defect. In each case Claude reports what the transcription says and asks for the original page.

## 2.7 Project protocols (optional)

For a Project whose documents you return to often, Claude can write a *project protocol* on request. This is a file named `pdf_protocol_<project>.md` that caches the facts measured for each document:

- its container and text-layer failure modes;
- its page offset;
- the glyph mappings verified on the image;
- project-specific notational traps;
- a machine-readable inventory with hashes, sizes, page counts and a prose fingerprint.

The protocol stores how to *read* the files, not what they *say*.

On the first use of the skill in each session, Claude runs a staleness check against the inventory:

| Exit | Meaning and action |
|---|---|
| 0 | Up to date: the cached facts are used. |
| 1 | Stale: Claude reports which files were added, removed or changed, reads them fresh, and offers an update. |
| 3 | Only *probable rendition swaps*: Claude asks whether the files were edited (see below). |

A Project file can be presented in different containers in different sessions, for example as a PDF in one session and as a paged ZIP in the next, without the user changing anything. The check recognises every recorded rendition. It flags an unknown one as a *probable* swap only when the page count and prose fingerprint agree, and you decide whether it is the same document. Source files and transcriptions never make a protocol stale.

Claude cannot write to project knowledge, so the protocol is handed over as a download. You upload it to the Project and add a pointer to the Project instructions, for example:

```
PDF reading: follow the academic-pdf-reading skill. Project-specific facts are in
pdf_protocol_<project>.md in project knowledge; run its staleness check on first use
each session.
```

### 2.8 `triage.py`

The script *measures*; it never decides what a glyph means. Its subcommands:

| Subcommand | Purpose |
|---|---|
| `tools` | Lists which helper tools exist in the current environment. |
| `profile FILE...` | Reports container, channels, text-layer signals, page-offset candidates and notes. |
| `inventory FILE...` | Builds the inventory block for a project protocol. |
| `check INVENTORY` | Runs the staleness check (exit codes 0 / 1 / 3; 2 for usage errors). |
| `text FILE N` | Prints the text of page N, with `--escape` to make control bytes visible. |
| `grep PATTERN FILE...` | Searches by regex and reports hits as *file:page:line*. |
| `extract FILE --out DIR` | Unpacks a paged ZIP, or writes one text file per PDF page. |
| `crop SRC --page N` | Renders a page image, optionally cropped and upscaled. |

## 3 Requirements

- A Claude plan with skills available and **code execution** enabled in Claude's settings.
- Python 3.8 or newer, which the code-execution environment provides.
- Optional tools, used when present: poppler-utils (`pdfinfo`, `pdftotext`, `pdftoppm`, `pdffonts`, `pdfimages`), Pillow and pypdf. Without them the skill falls back to alternatives (pypdf or pdfplumber for text, PyMuPDF for rendering, viewing whole pages instead of crops) and tells you when confidence is reduced.
- No network access is needed.

## 4 Installation

1. **Package the skill.** The skill is a folder named `academic-pdf-reading` containing `SKILL.md`, `references/` and `scripts/`. If you received it as a ZIP, use that file as it is. Otherwise, zip the folder so that the archive contains the folder itself, with `SKILL.md` directly inside it:

   ```
   zip -r academic-pdf-reading.zip academic-pdf-reading
   ```

2. **Enable code execution** in Claude's settings if it is not already on. Skills run their scripts in the code-execution environment.
3. **Upload.** Open Customize → Skills, choose to upload a skill, and select the ZIP.
4. **Turn it on.** Check that *academic-pdf-reading* appears in the list and is enabled.
5. **Verify.** In a new chat, attach a paper and ask, for example: *"Profile this file: what container is it, and what does its text layer get wrong?"* Claude should read the skill and run `triage.py profile`.

> **Updating.** Keep only one copy installed. Remove the previous version and upload the new ZIP in its place. The installed version is recorded in the `metadata.version` field of `SKILL.md`. Existing project protocols remain usable; Claude offers to update one when the staleness check or a file change calls for it.

**Without installing the skill.** `scripts/build_bundle.py` generates a single Markdown file, `academic-pdf-reading.md`, that contains `SKILL.md`, the project-protocol reference and `triage.py`.

Upload it to a Project's knowledge to use the procedure where the skill cannot be installed. The bundle is generated output: edit the skill folder and rebuild it, never edit the bundle by hand.

## 5 Usage

You do not need to name the skill. Typical requests that engage it:

- *"What is the sign convention in Eq. (12) of the attached paper?"*
- *"Quote the definition in Section 3 exactly, with the page number."*
- *"Does the index in Eq. (4.17) carry an overbar or not?"*
- *"This file is called .pdf but my viewer will not open it. What is it?"*
- *"Create a reading protocol for this Project's documents."* or *"I added two papers; update the protocol."*

Answers cite the printed page and equation tag and say how each reading was obtained: *verified at p. N*, *provisional*, *text-layer reading, unverified*, or *transcription, p. N*.

## 6 Limitations

- **Image verification costs context.** Checking many equations against page images in one conversation is expensive. For documents you return to often, converting them once with pdf-to-markdown is cheaper.
- **Text-only files cannot be verified.** When no page image exists, mathematics stays labelled unverified, and Claude cross-checks it against another source or re-derives it where possible.
- **OCR text has no deterministic repair.** Even prose from a scanned document is checked on the image before it is quoted.
- **Fingerprints identify documents, not versions.** A revision that changes only a few equations can look identical to the original, which is why probable rendition swaps always end in a question to you.
- **Protocols are uploaded by you.** Project files are read-only to Claude, so every new or updated protocol is delivered as a download.

## 7 Working with pdf-to-markdown

The two skills are designed as a pair. **pdf-to-markdown** spends the image-reading effort once, at conversion time, and produces a transcription with exact LaTeX mathematics. **academic-pdf-reading** recognises that transcription and reads it as source. It also handles the original PDFs, text extractions and paged archives that have not been converted. In a project protocol, a document replaced by its transcription is reported as *converted*, and its entry shrinks to a short record of the transcription.

# 8 Package contents

| File | Purpose |
|---|---|
| `SKILL.md` | The reading procedure: policy, triage, access recipes, failure modes, page numbering, workflow. |
| `references/project_protocol.md` | How to create, verify, hand over and update a project protocol, with a template. |
| `scripts/triage.py` | Deterministic measurements: profiling, inventory, staleness check, text, grep, extract, crop. |
| `scripts/build_bundle.py` | Generates the standalone single-file bundle. |